

Ultimate Cell-by-Cell Master Blueprint for Machine Learning Execution

This document serves as the authoritative technical protocol for executing Machine Learning projects through a disciplined, cell-by-cell methodology. It enforces a strict separation between the **System** (the loop, guardrails, and checkpoints) and the **Subject** (the specific ML deliverable). The **Pilot** (Human) remains the sole authority for execution and reflection, while the **Agent** (AI) is restricted to proposing, pre-filling from source context, and implementing to contract.

1. Phase 1: Pre-Flight System Setup (The Bubble)

1.1 Environment Isolation & Directory Hierarchy

All development **shall** occur within a "Bubble"—a project-local isolated environment. One deliverable equals one isolated environment. **Standard Project Structure:**

```
.
├── .venv/                # Layer 0: Isolated environment
├── docs/
│   └── adr/              # ADRs: 0001-xxxx.md
├── specs/
│   ├── Product_Spec.md  # Why/What (Precedence: High)
│   └── Tech_Spec.md     # How (Precedence: Low)
├── src/                  # Implementation logic
├── tests/                # Layer 3: Contract tests
├── progress.md           # Living State: Snapshot & Logs
└── checkpoints.md       # Gate reference & Resume Protocol
```

1.2 Layer 1: The Setup Gate

The Agent **shall** write a setup script that returns exit 0 **only** if all real dependencies are present. **Setup Gate Checklist:**

- **Isolated Environment:** Active (Layer 0).
- **Dependency Diff:** A "Diff actual-vs-declared" pass identifies any packages used in code but missing from requirements.txt.
- **Dependency Installation:** All declared dependencies are present in the .venv.
- **Environmental Integrity:** Necessary API keys, data paths, and Python versions are validated.

1.3 Initial Contract (Step 0 & Checkpoint C0)

Before any construction, the Agent **shall** perform a "Step 0" comprehension pass. The Agent **must** pre-fill these components from the source context but **is strictly forbidden** from advancing past the C0 gate without Pilot confirmation.

- **Purpose:** The core objective and the "Why."
- **Inputs:** Source data, provided libraries, and prerequisite starter code.

- **Outputs:** Exact form of final deliverables (e.g., serialized model, evaluation report).
- **Flow:** High-level architectural connection between inputs and outputs.
- **Definition of Done (DoD):** Concrete, checkable criteria for completion.
- **Scope & Constraints:** Explicit boundaries to prevent over-engineering.

1.4 Specification Precedence

The "Spec Set" defines the standing orders. Conflict resolution is governed by the following hierarchy: | Document | Focus | Precedence || ----- | ----- | ----- || Product_Spec.md |

Why/What: Intent, user stories, metrics, and acceptance criteria. | **Wins on conflict.** ||

Tech_Spec.md | **How:** Guardrail layers, library choices, and naming conventions. | Amended to match Product Spec. |

2. Phase 2: Cell-by-Cell Execution Engine (The Builder Loop)

2.1 The P-I-O-F Planning Model (Layer 2 / Checkpoint C1)

Every individual code unit or notebook cell **shall** be governed by the P-I-O-F protocol.

Generation of implementation code is forbidden until the Pilot approves the C1 gate.

1. **Purpose:** The specific goal of the unit.
2. **Inputs:** Variables or states required from the preceding context.
3. **Outputs:** The specific result (e.g., a scaled feature set, a training history).
4. **Flow:** The internal logic or mathematical transformation.

2.2 The Amendment Rule (Contract Drift Protocol)

Understanding at Step 0 is a snapshot; building is empirical. If reality contradicts the plan:

1. **Pause — do not silently patch.** A missing rule is a contract failure, not a code failure.
2. **Loop back to C0.** Amend the P-I-O-F, DoD, or Scope. State clearly what changed and why.
3. **Pilot re-approves.** Work continues only after the Pilot signs off on the amended contract.

2.3 Contract-First Development (Layer 3 & Layer 4)

- **Layer 3 (Tests as Contract):** Use TDD. Write the assertion *before* the logic. A "known-bad" input **must** be fed to the test to ensure it fails. A test that cannot fail is not a valid gate.
- **Layer 4 (Pre/Post Assertions):** In Machine Learning, transformations are often silent. Every feature engineering unit **must** include integrity checks (e.g., `assert df['col'].notnull().all()`) before and after the transform.

2.4 The Failure Protocol (Checkpoint C3 & ADRs)

When a unit fails, the Agent **shall** log the symptom context (C3) **before** proposing a fix.

- **Any Decision Record (ADR):** If a fix requires a non-obvious choice (e.g., swapping `StandardScaler` for `MinMaxScaler` or pinning a library version), the Agent **shall** create a numbered ADR in `docs/adr/NNNN-title.md`.
- **ADR Format:** Status, Context, Decision, and Consequences.

3. Phase 3: Repository Integrity & Delivery

3.1 Living State & Resumability

The project state is maintained in progress.md. It **shall** follow this structure to allow for "Cold Starts":

```
## Snapshot
- Current Stage: [e.g., Data Preparation]
- Last Checkpoint: [e.g., C2]
- Next Single Action: [State one specific action]

## Unit Log
- [Unit Name] | [Commit Hash] | [Verification Status]

## Failure & Amendment Logs
- [Failure Symptom] | [ADR Ref] | [Resolution]
- [Contract Drift] | [C0 Amendment Date]
```

Resume Protocol (Cold Start): The Agent **shall** read progress.md, cross-check the git history, run the Layer 1 Setup Gate, and state the next action before requesting Pilot input.

3.2 Atomic Commits (Checkpoint C2)

"One unit = one commit." Every successful cell execution triggers a C2 record including the unit name, the testing performed, and the Pilot's validation status.

3.3 Layer 5: Objective Proof (CI/CD)

A local pass is subjective. A repository pass is objective proof of reproducibility.

- **Tier 1:** CI on push (Clean env, install, setup gate, contract tests).
- **Tier 2:** Local pre-commit hooks enforcing C2 formats.
- **Tier 3:** Notebook execution enforcement (e.g., nbmake or nbconvert --execute).

3.4 The C4 Reflection Interview

Once DoD is met and CI is green, the Agent **shall** conduct a dynamic interview.

- **Dimensions:** Comprehension, territory novelty, verification methods, contract drift, and local-vs-objective proof.
- **Verbatim Requirement:** reflections.md **must** be a verbatim Q&A transcript. The Agent **shall not** paraphrase, edit, or "improve" the Pilot's answers.

4. Machine Learning Workflow Integration

4.1 Mapping ML Phases to the Loop

Every sub-step of the ML workflow **shall** be treated as a "Unit" in the Builder Loop, requiring its own P-I-O-F, tests, and C2 commit.

1. **Data Collection & Understanding:** Defining inputs and exploring distributions.

2. **Data Preparation:** Cleaning and Feature Engineering. This consumes **80% of project time** . Each transformation (e.g., encoding a column) is a Layer 4 assertion opportunity.
3. **Model Training & Evaluation:** Selecting algorithms and measuring against metrics.
4. **Deployment & Monitoring:** Moving to production and tracking drift.

4.2 Learning Type Categorization

Learning Type, Data Characteristic, Core Goal

Supervised Learning, Labeled data (Input-Output pairs), Prediction of outcomes for new data.

Unsupervised Learning, Unlabeled data, Discovery of hidden patterns/groupings.

Reinforcement Learning, Environment interactions, Maximizing rewards via trial and error.

5. Cursor Initiation Prompt (The "Magic Cell")

Copy and paste the following block into the first cell of your notebook to initialize the Build Engine:

```
# [SYSTEM INITIALIZATION: ML BUILD ENGINE]
```

As an Expert Information Synthesizer and Lead Systems Architect, adopt the "Agent" persona.

1. ****Context Pass:**** Read all project documentation, source materials, and instructions.
2. ****Initialize State:**** Create `progress.md` and `checkpoints.md` per the "Living State" protocol.
3. ****Execute Step 0:**** Generate the initial P-I-O-F, Definition of Done (DoD), and Scope/Constraints pass for the entire project.
4. ****Session-Start Protocol:**** Run the Layer 1 Setup Gate (including dependency diff) and report findings.
5. ****Human Gate:**** WAIT for Pilot confirmation (Checkpoint C0) before any further action.

****Operational Rules:****

- Follow the "One unit = one commit" (C2) rule.
- Log failures (C3) and decisions (ADRs in docs/adr/) before proposing fixes.
- Adhere to the Amendment Rule: Pause and loop back to C0 on contract drift.
- You are forbidden from self-verifying code; only the Pilot may run cells.
- Generate only renderable Markdown text. No graphs, mermaid, or images.

****Awaiting Step 0 output based on source context...****